

课程笔记

课程笔记：MIT RES.9-009 05: Blox 与连接 (Blox and Connections in Neuroblox)

五道口纳什 & Codex

2026 年 4 月 15 日



视频作者/频道：MIT Video Productions / MIT OpenCourseWare

发布日期：2025-01-14 (YouTube upload: 2025-04-10)

视频时长：19:18

视频链接：<https://www.youtube.com/watch?v=8XcN9j5njgg>

目录

1 课程定位与本讲主线

这一讲是本课程中最后一场偏“软件架构”的技术课。讲者明确把目标放在两件事上：第一，让你看懂 Neuroblox 的代码结构，知道它如何把一个神经模型拆成可以组合、可以检查、可以扩展的模块；第二，让你具备写出新 Blox 和新连接规则的能力，而不是只会调用现成模型。

从课程设计上讲，这一讲是从“用 Neuroblox 跑模型”走向“为 Neuroblox 写模型”的分水岭。前几讲中，我们已经见过单个神经元、神经团块（neural mass）、外部输入源以及简单电路；而本讲的重点是把这些对象抽象成图（graph），再用 Julia 的类型系统和 multiple dispatch 把“如何连接”“如何响应事件”“如何向后兼容”编码进库本身。

如果把 Neuroblox 的建模方式压缩成一句话，那就是：

模型 = Blox 节点 + Connection 边 + 图上的系统编译

这里的“节点”不是普通数据结构，而是带状态、参数、输入输出和事件的可计算对象；“边”也不是简单的连线，而是会被编译成具体符号方程的连接语义。后面几节的所有代码，基本都围绕这个抽象展开。

本讲的核心抽象

Blox 负责“一个对象内部是什么”，connection rule 负责“两个对象之间如何交换信息”，graph 负责“整个模型如何被组装”，而 multiple dispatch 负责“当对象类型变化时，系统应该自动选择哪一套连接或回调逻辑”。

1.1 本章小结

本讲不是在补神经科学背景，而是在训练你读懂 Neuroblox 的工程骨架。只要把“对象、连接、图、调度”这四层抓住，后面的自定义 neuron、circuit、source 和大规模电路就都能顺着同一条逻辑扩展下去。

2 Blox 类型树与对象检查

Neuroblox 把大量建模对象统一称为 Blox。这个名字背后真正重要的不是“组件很多”，而是它们遵循一棵清晰的类型树：顶层是 AbstractBlox，其下有 Neuron、NeuralMass、CompositeBlox、SpikeSource 等分支。讲者强调，这种设计让 Julia 的类型系统可以发挥作用：当你写出一个新的子类型，如果它继承了 Neuron，许多通用连接规则和绘图函数就会自动可用。

这种设计的直接收益是“少写胶水代码”。例如，很多连接规则并不是针对某一具体模型单独写的，而是针对某一类对象写的；只要你的新 Blox 落在正确的类型分支上，库里已经实现的默认行为就能继续工作。这比把每个模型都写成互不相干的脚本更可维护。

```
1 using Neuroblox
2 using OrdinaryDiffEq
3
4 @named lif = LIFNeuron()
5
6 typeof(lif)
7 lif isa Neuron
8 unknowns(lif)
```

```

9 parameters(lif)
10 inputs(lif)
11 outputs(lif)
12 discrete_events(lif)
13 equations(lif)

```

Listing 1: 检查一个 LIFNeuron 的典型接口

上面的接口展示了 Neuroblox 借鉴 ModelingToolkit 的方式：一个 Blox 不只是一个“函数名”，而是一个可以被 introspect 的符号系统。`unknowns` 给出未知量，`parameters` 给出参数，`inputs` 和 `outputs` 给出数据流方向，`discrete_events` 给出离散事件，`equations` 则把对象内部动力学直接展开成方程。

以 LIFNeuron 为例，讲者展示的方程输出可以概括为两条：

$$\frac{dV}{dt} = \frac{I_{in} + (E_m - V)/R_m + jcn(t)}{C}, \quad \frac{dG}{dt} = -\frac{G}{\tau}.$$

其中， V 是膜电位， G 是一个与突触传递有关的内部状态， I_{in} 是外部注入电流， $jcn(t)$ 是连接项， C 、 R_m 、 τ 、 E_m 都是参数。这里的关键不是某个单独模型，而是 Neuroblox 会把“连接项”当作可累积的输入槽位来处理。

为什么要先 inspect 再连接

在把一个 Blox 接入大模型之前，先看它的 `equations` 和 `inputs/outputs` 是必要步骤。否则你很容易把一个输出变量误当成输入变量，或者把一个需要离散事件触发的模型误接成连续驱动。

2.1 本章小结

Blox 的本质是“可 introspect、可组合、可扩展”的符号模型对象。只要对象落在正确的类型树分支上，Neuroblox 就能自动继承相当多的默认行为，这也是它比手写脚本更适合做大型计算神经科学模型的原因。

3 连接规则与连接语义

理解了 Blox 之后，下一步就是理解它们如何互相连接。Neuroblox 里有两套紧密相关但用途不同的接口：`connection_equations` 用来打印或返回“这条边会生成什么方程”，`connection_rule` 则会把方程、权重以及连接类型一起展开给你看。讲者特别强调，后者更适合做调试和理解默认行为，因为它能把隐式参数一起暴露出来。

对于两个神经元 Blox，最基础的连接可以理解为一个带权重的信号传递。讲者示范了 LIFNeuron 到 IFNeuron 的连接：在最简单的 `basic` 规则下，连接项就是由前一个神经元的内部状态驱动后一个神经元的输入槽位；而在 `psp` 规则下，还会多出一个与后突触电位相关的驱动力因子。

对应到符号层面，默认连接大致可以写成：

$$jcn_{ifn}(t) = w_{lif \rightarrow ifn} G_{lif}(t),$$

而 `psp` 规则会把后突触电位差也乘进去：

$$jcn_{ifn}(t) = (E_{syn,lif} - V_{ifn}(t)) w_{lif \rightarrow ifn} G_{lif}(t).$$

这里最值得注意的是，连接项不再只是“一个权重”，而是“一个动力学项”。这意味着连接规则不仅决定连接强度，还决定连接对后突触变量的作用形式。

```
1 @named ifn = IFNeuron()
2
3 connection_equations(lif, ifn, weight=1, connection_rule="basic")
4 connection_rule(lif, ifn, weight=1, connection_rule="psp")
```

Listing 2: 查看两种默认连接语义

讲者还指出，`weight` 和 `connection_rule` 都可以省略；如果省略，Neuroblox 会给出默认值的提示。这个设计很实用，因为它把“库的默认行为”显式地告诉你，减少了黑箱感。

连接语义不是纯粹的图论边

在 Neuroblox 里，边不是单纯的“有没有连上”，而是要被解释成方程、权重和事件的组合。换句话说，图结构只是入口，真正进入 ODE/SDE 系统时，边会被翻译成可求解的符号项。

3.1 本章小结

连接规则决定的是“信息如何流动”，不是“节点之间有没有边”。`connection_equations`、`connection_rule` 和默认规则共同构成了 Neuroblox 的连接语义层，也是后面写自定义连接时必须遵守的接口约定。

4 图到系统：从 MetaDiGraph 到 ODEProblem

当 Blox 和连接都定义好之后，Neuroblox 还需要一个总控对象把它们变成可求解系统。这里的关键容器是 `MetaDiGraph()`。讲者给出的思路非常直接：每个顶点是一个 Blox，每条边是一条连接规则，最后再调用 `system_from_graph` 把图编译成一个统一的动力系统。

```
1 g = MetaDiGraph()
2 add_edge!(g, lif => ifn, weight=1)
3
4 @named sys = system_from_graph(g)
5 prob = ODEProblem(sys, [], (0, 200.0))
6 sol = solve(prob, Tsit5())
```

Listing 3: 把两个 neuron 连接成一个可求解系统

这段代码背后的工作量其实很大。`system_from_graph` 不只是把图“拼接”起来，它还会处理命名空间、收集连接项、做结构化简化，并把结果变成标准的 `ODESystem`。所以从用户角度看，你写的是图；从求解器角度看，它拿到的是一个规范化的微分方程系统。

讲者把这件事概括成一句很重要的话：Neuroblox 的模型“从图开始”。这个说法不是比喻，而是实现方式本身。你用图来定义结构，用 `edge` 的 `metadata` 定义动力学细节，然后让系统编译器完成最后一步。

为什么这一步是 Neuroblox 的核心

如果没有 `system_from_graph`，每个模型都得自己手写方程拼接、变量重命名和连接项累积；有了它，模型结构就可以像搭积木一样组合、替换和扩展。

4.1 本章小结

MetaDiGraph 到 ODEProblem 的转换，是 Neuroblox 从“结构描述”走向“数值求解”的关键桥梁。这个桥梁的存在，使得大模型可以像小模型一样被统一处理。

5 自定义 Blox: IzhNeuron 的写法

本讲最重要的工程示范之一，是把前面课程中手工写过的 Izhikevich 神经元封装成一个新的 Blox。讲者想传达的不是“这个模型有多复杂”，而是“一个自定义 Blox 在 Neuroblox 里到底长什么样”。

最小的自定义 Blox 结构非常简单：它至少要保存两类信息，`system` 和 `namespace`。前者存放动力学系统本身，后者用于层级模型中的命名空间管理。对单个原子 Blox 来说，`namespace` 可以先保持为 `nothing`；但一旦进入层级模型，它就变成避免变量冲突的必要机制。

```
1 struct IzhNeuron <: Neuron
2     system
3     namespace
4
5     function IzhNeuron(; name, namespace=nothing,
6                         a=0.02, b=0.2,
7                         V_reset=-50, d=2, threshold=30)
8         sts = @variables V(t)=-65 [output=true] u(t)=-13 jcn [input=true]
9         params = @parameters a=a b=b V_reset=V_reset d=d theta=threshold
10        eqs = [D(V) ~ 0.04 * V^2 + 5 * V + 140 - u + jcn + 5,
11              D(u) ~ a * (b * V - u)]
12        event = (V > theta) => [u ~ u + d, V ~ V_reset]
13        sys = System(eqs, t, sts, params; name=name, discrete_events=event)
14        new(sys, namespace)
15    end
16 end
```

Listing 4: 一个最小可用的 IzhNeuron 自定义 Blox

这里最值得读懂的有三点。第一，`@variables` 明确声明了状态变量，并通过 `[output=true]` 和 `[input=true]` 标记数据流方向；第二，`@parameters` 把参数做成可命名、可传值的符号对象；第三，`discrete_events` 把尖峰重置机制显式地写进系统，而不是藏在外部回调里。

对应的动力学可写成：

$$\dot{V} = 0.04V^2 + 5V + 140 - u + jcn + 5, \quad \dot{u} = a(bV - u),$$

并在 $V > \theta$ 时触发

$$u \leftarrow u + d, \quad V \leftarrow V_{\text{reset}}.$$

讲者特别提醒，`jcn` 这个输入槽位不需要你手动给定初值；Neuroblox 会自动把它初始化成 0，并在后续连接中累积所有输入项。

输入与输出标签的实际意义

`input=true` 不只是元数据，它决定了这个变量会不会被连接规则自动写入；`output=true` 也不只是注释，它决定了别的 Blox 是否能把它当作默认输出来源来读取。

5.1 本章小结

自定义 Blox 的核心不是“写一个 Julia struct”这么简单，而是把状态、参数、输入输出标记、离散事件和命名空间一起放进同一个可编译对象里。只要这五样齐全，你就已经进入 Neuroblox 的标准扩展路径了。

6 自定义连接规则与回调

有了自定义 Blox，下一步自然是让它和别的对象正确连接。讲者在这里展示了 multiple dispatch 的真正价值：你不需要改库本身，只要为更具体的源/目标类型写一个新方法，Neuroblox 就会自动选中它。

第一个例子是把 LIFNeuron 接到 IzhNeuron 上。默认情况下，Neuroblox 会退回到通用连接；但一旦你定义了更具体的 connection_equations(source::LIFNeuron, destination::IzhNeuron, weight; kwargs...)，系统就会优先调用它。

```
1 import Neuroblox: connection_equations
2
3 function connection_equations(source::LIFNeuron,
4                               destination::IzhNeuron,
5                               weight; kwargs...)
6     equation = destination.jcn ~ weight * source.G * (destination.V - source.E_syn)
7     return equation
8 end
```

Listing 5: 为更具体的类型组合定义连接方程

这个例子说明了两个关键点。其一，kwargs... 让接口可以接收更多扩展参数，因此方法不会轻易过时；其二，连接方程不仅可以简单地把前一个对象的输出搬过来，也可以引入后突触变量 destination.V，从而把连接做成更符合生理语义的驱动力项。

讲者接着演示了一个更灵活版本：在从 IzhNeuron 到 LIFNeuron 的连接里，额外引入 const_current，让同一条方法可以带不同的常数偏置。对应代码可以写成：

```
1 function connection_equations(source::IzhNeuron,
2                               destination::LIFNeuron,
3                               weight; const_current=1, kwargs...)
4     equation = destination.jcn ~ weight * source.V + const_current
5     return equation
6 end
```

Listing 6: 用额外 keyword 参数扩展连接方程

最后，讲者把“连续连接”与“尖峰回调”区分开来：前者通过方程项把信号并入系统，后者通过 connection_callbacks 在事件条件满足时更新状态。对 Izhikevich 这样的模型，尖峰通常意味着要修改下游神经元的导通变量，而不是简单地加一个常数。

```
1 import Neuroblox: connection_callbacks
2
3 function connection_callbacks(source::IzhNeuron,
4                               destination::LIFNeuron;
5                               spike_conductance=1, kwargs...)
```



```

6     spike_affect = (source.V > source.theta) =>
7         [destination.G ~ destination.G + spike_conductance]
8     return spike_affect
9 end

```

Listing 7: 用 callback 把尖峰事件映射到下游变量更新

这就是本讲工程部分最重要的结论：连续连接和离散回调不是互斥的，而是分别适配不同生理现象的两种扩展方式。你在写自定义 Blox 时，往往需要两者一起定义，才能让模型既有连续亚阈值动力学，又能在尖峰时刻正确传播影响。

6.1 本章小结

multiple dispatch 让 Neuroblox 的扩展不是“修改老代码”，而是“为新类型添加新方法”。连接方程负责连续动力学，callback 负责离散事件；两者配合，才能把一个新模型真正接进库里。

7 挑战任务与工程实践

课程页面最后给了三个方向的挑战任务，用来检验你是否真的理解了本讲的扩展模式。第一类是把 Morris-Lecar 模型做成新的 Blox，并为它和 HHNeuronExciBlox、HHNeuronInhibBlox 写连接规则；第二类是实现一个 generalized leaky integrate-and-fire 网络，验证它在更大规模网络里的可复用性；第三类是实现一个 Hopf network 的随机动力学版本，看看同样的 Blox 架构能否承载完全不同的神经动力学家族。

这些题目的共同点不是“模型名字不同”，而是它们都在考你同一套工作流：定义类型、声明状态和参数、写出连接语义、让图编译成系统，再用适当的求解器检查行为。换句话说，题目本身就是对本讲架构能力的回放。

讲者还给出了非常实用的工程建议：如果你想快速了解一个新 Blox，可以在 Julia REPL 里输入 ?TypeName 看文档，或者用 @edit TypeName() 直接打开源码。对于一个正在扩展的库，这种“先查文档，再看源码，再补 dispatch”的工作流，比盲目试错有效得多。

适合复习时反复问自己的三个问题

这个对象属于哪一层类型树？它的输入、输出和事件分别是什么？如果我要让它和另一类 Blox 连接，我应该写连接方程还是 callback？

7.1 本章小结

挑战题的价值不在于“做完所有作业”，而在于逼你把本讲的抽象真正落到代码里。只要你能独立完成一个新 Blox 和一条新连接规则，就说明你已经理解了 Neuroblox 的扩展范式。

8 总结与延伸

这一讲把 Neuroblox 的工程骨架完整地拆开了：Blox 是对象本身，connection rule 是对象间语义，graph 是全局结构，multiple dispatch 是扩展机制。更重要的是，这四层不是分离的，而是彼此咬合的：类型树决定默认行为，连接规则决定动力学，图决定系统组合方式，而求解器只需要最终的规范化系统。

如果把本讲压缩成一个可执行的检查清单,它会是这样的:先 inspect 一个已有 Blox, 确认变量和事件;再看它的连接规则,理解连续项和回调项;然后把几个 Blox 放进图里,用 `system_from_graph` 验证整个系统;最后再写自己的子类型和更具体的 `dispatch`。这个顺序并不是教学上的偶然安排,而是实际做模型时最稳妥的开发路径。

8.1 拓展阅读

如果你想继续沿着本讲的挑战题往下走,最值得回看的资料是: Morris & Lecar (1981) 的电压振荡模型, Lorenzi et al. (2023) 的 generalized leaky integrate-and-fire 多层均值场模型, 以及 Ponce-Alvarez & Deco (2024) 的 Hopf whole-brain model。它们分别对应了“新 neuron 类型”“新网络族”“新随机动力学”的三种扩展方向。

8.2 本章小结

本讲教会你的不是某个单独模型,而是“如何让新模型融入 Neuroblox”。如果你能把新 Blox 的定义、连接规则、callback 和图编译四步串起来,后面的课程内容就只是在这个架构上不断增加生物学层级而已。